

CAMPUSKEY

Secure University Access System

Complete Project Summary

Project Overview

CAMPUSKEY is a secure university access system with multi-factor authentication, role-based access control (Admin, Professor, Student), and comprehensive security features.

★ Key Features Implemented

Authentication System

-  Email-based OTP verification (code sent via SMTP)
-  Simulated Biometric authentication
-  Simulated RFID card authentication
-  Username-based login (email not stored in database)
-  Session management with Flask-Login

Role-Based Access Control

-  **Admin:** System administration, user management, login logs, grades overview
-  **Professor:** Course management, grade assignment, student management
-  **Student:** View grades, view courses, academic portal

Separate Dashboards per Role

- `/admin/dashboard` - Admin dashboard with system overview
- `/professor/dashboard` - Professor dashboard with courses and grading
- `/student/dashboard` - Student dashboard with grades and courses

Security Features

-  Report an Issue
-  Security Guidelines (comprehensive university policy)
-  Recent Activity (with IP and timestamp details)
-  Device Security
-  Account Protection Level
-  View Access History
-  Generate Access Code (random passcode generator)

Database Models

```
User: username, role, otp_secret (email optional, not stored)
Course: course code, name, professor assignment
Grade: student grades with percentage and letter grade
LoginAttempt: login history tracking
ActiveSession: session management
EmailVerificationCode: OTP code storage
```

Dependencies

```
Flask==2.3.3
Flask-Login==0.6.3
Flask-SQLAlchemy==3.0.5
pyotp==2.8.0
qrcode==7.4.2
Pillow==10.0.0
python-dotenv==1.0.0
gunicorn==21.2.0
twilio==8.10.0
```

Setup & Installation

Quick Start

```
# Navigate to project directory
cd "/Users/fortuneogulewe/Downloads/project security"

# Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure email (optional)
source email_config.sh

# Run the application
python app.py

Access: http://localhost:5001
```

Demo Users

Role	Username
Admin	admin, fortune
Professor	professor, abdulsalam
Student	student, hunter, philip, haoyun, liandro, jamie, felix, david, james, kalpan, nidhi, carson, macken

Total: 19 users

📁 Project Structure

```
project security/
  app.py                # Main Flask application
  config.py             # Configuration settings
  models.py            # Database models
  auth.py              # Authentication utilities
  email_service.py      # Email OTP sending
  requirements.txt      # Python dependencies
  runtime.txt          # Python version
  email_config.sh       # Email configuration script
  instance/
    campuskey.db        # SQLite database file
  templates/           # HTML templates
  static/              # CSS and JavaScript
  venv/                # Virtual environment
```

🔑 Authentication Flow

1. 👤 User enters username and email on login page
2. 📧 Clicks "Send Code" → Verification code sent to email
3. 🔑 Enters code → System verifies code
4. ✅ User authenticated → Redirected to role-specific dashboard

Note: Email is NOT stored in database - only used for code delivery. Users are identified by username only.

🔗 Key Routes & Endpoints

Public Routes

- / - Home (redirects to login)
- /login - Login page

Protected Routes (Requires Login)

- /dashboard - Redirects to role-specific dashboard
- /admin/dashboard - Admin dashboard
- /professor/dashboard - Professor dashboard
- /student/dashboard - Student dashboard

API Endpoints

- /api/send-email-code - Send email verification code
- /api/biometric-login - Simulate biometric login
- /api/rfid-login - Simulate RFID login
- /api/generate-passcode - Generate random passcode

Common Workflows

Adding a New User (Admin)

1. Login as admin
2. Go to "Manage Users" in sidebar
3. Click "Add User"
4. Enter username and select role
5. Save

Professor Giving Grades

1. Login as professor
2. Go to "Give Grades"
3. Select course and student
4. Enter grade value and percentage
5. Submit

Project Completion Status

Feature	Status
Email-based OTP authentication	✔ Complete
Role-based dashboards	✔ Complete
User management system	✔ Complete
Course and grade management	✔ Complete
Security features and guidelines	✔ Complete
Login logging and activity tracking	✔ Complete
Access history and device security	✔ Complete
Random passcode generator	✔ Complete
Responsive UI with role-based navigation	✔ Complete

Important Notes

-  **Admin Access:** Must use username `admin` (not just admin role) for full admin features
-  **Email Storage:** Email addresses are NOT stored - only used for code delivery
-  **Database Location:** `instance/campuskey.db` (SQLite)
-  **Port:** 5001 (to avoid AirPlay conflict)
- **Authentication:** Email OTP is primary method; biometric/RFID are simulated

>_ Quick Reference

Login Credentials

- **Admin:** username `admin`, any email for code
- **Professor:** username `professor`, any email for code
- **Student:** username `student`, any email for code

File Locations

-  **Database:** `instance/campuskey.db`
-  **Config:** `config.py`
-  **Email Config:** `email_config.sh`
-  **Main App:** `app.py`

Project Status:  **Complete & Functional**

Last Updated: November 2024